

PANDUAN & PENJELASAN DIAGRAM SISTEM

Cloud-Based Data Processing & Monitoring Platform

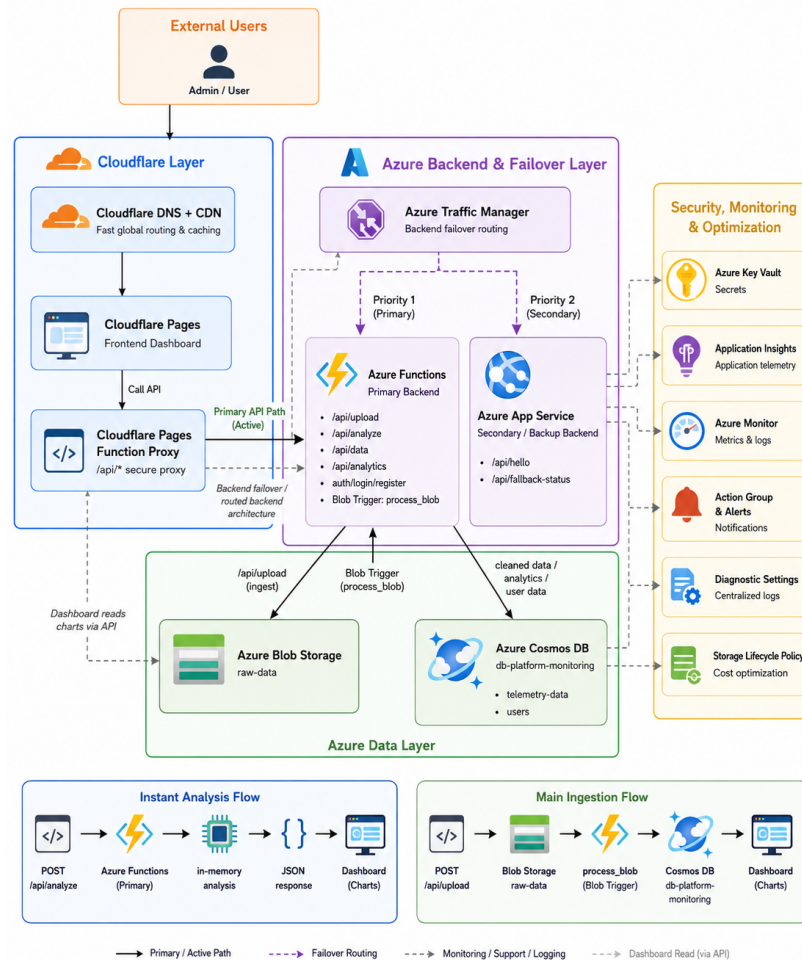
Kelompok 11 — Tugas Akhir Cloud Computing 2026 | 5 Diagram Arsitektur

Dokumen ini menyajikan penjelasan mendalam atas setiap diagram arsitektur dan diagram urutan (sequence diagram) yang digunakan dalam proyek ini. Setiap diagram dijelaskan komponen demi komponen, anak panah demi anak panah, menggunakan bahasa yang mudah dipahami oleh siapa pun — termasuk pembaca yang tidak memiliki latar belakang teknis.

Daftar Diagram dalam Dokumen Ini

No.	Judul Diagram	Jenis
1	Arsitektur Keseluruhan Sistem (Overall Architecture)	Arsitektur
2	Alur Failover Azure Traffic Manager	Alur Failover
3	Sequence Diagram: Main Ingestion Flow (POST /api/upload)	Sequence
4	Sequence Diagram: Instant Analysis Flow (POST /api/analyze)	Sequence
5	Sequence Diagram: Alur Login & Autentikasi	Sequence

Arsitektur Keseluruhan Sistem (Overall Architecture)



Gambar 3.1 — Diagram Arsitektur Keseluruhan Sistem Cloud-Based Data Processing & Monitoring Platform Kelompok 11

Gambaran Umum Diagram

Diagram ini adalah peta besar dari keseluruhan sistem. Diagram menggambarkan bagaimana seluruh komponen — mulai dari pengguna di browser hingga basis data di cloud — saling terhubung dan berinteraksi satu sama lain. Sistem dibagi menjadi empat lapisan utama yang terlihat jelas pada diagram: Cloudflare Layer, Azure Backend & Failover Layer, Azure Data Layer, serta kolom Security, Monitoring & Optimization.

Lapisan 1 — Cloudflare Layer (Kiri)

Lapisan ini adalah 'gerbang depan' sistem yang pertama kali disentuh oleh pengguna. Seluruh komponen pada lapisan ini dikelola oleh Cloudflare, bukan Azure.

Cloudflare DNS + CDN menerima permintaan dari browser pengguna dan menyajikan halaman dashboard dari jaringan server global Cloudflare yang paling dekat dengan lokasi pengguna. Cloudflare Pages adalah platform hosting untuk berkas HTML, CSS, dan JavaScript dashboard. Cloudflare Pages Function Proxy adalah komponen terpenting di lapisan ini: ia mencegah semua permintaan ke path `/api/*`, menyisipkan Function Key Azure secara rahasia, lalu meneruskan permintaan ke Traffic Manager di Azure.

Lapisan 2 — Azure Backend & Failover Layer (Tengah Atas)

Lapisan ini adalah otak pemrosesan sistem. Azure Traffic Manager berdiri sebagai penjaga gerbang yang menentukan ke mana setiap permintaan API akan diarahkan.

Anak panah 'Priority 1 (Primary)' menunjukkan bahwa dalam kondisi normal, semua permintaan dikirim ke Azure Functions yang menjalankan Python 3.11 dan melayani semua endpoint utama (/api/upload, /api/analyze, /api/data, login, register, dsb.). Anak panah 'Priority 2 (Secondary)' menunjukkan jalur cadangan yang hanya aktif saat Azure Functions tidak tersedia, diarahkan ke Azure App Service yang menyediakan respons minimal (/api/hello dan /api/fallback-status).

Lapisan 3 — Azure Data Layer (Tengah Bawah)

Lapisan ini adalah 'gudang data' sistem. Azure Blob Storage (kontainer raw-data) menyimpan salinan berkas mentah yang diunggah pengguna. Azure Cosmos DB (db-platform-monitoring) menyimpan hasil pemrosesan dalam dua kontainer: 'telemetry-data' untuk rekaman data terproses, dan 'users' untuk data akun pengguna.

Anak panah kuning bertuliskan 'Blob Trigger (process_blob)' menggambarkan mekanisme event-driven: saat file baru masuk ke Blob Storage, Azure Functions secara otomatis dipicu untuk memproses berkas tersebut tanpa perlu perintah HTTP dari pengguna.

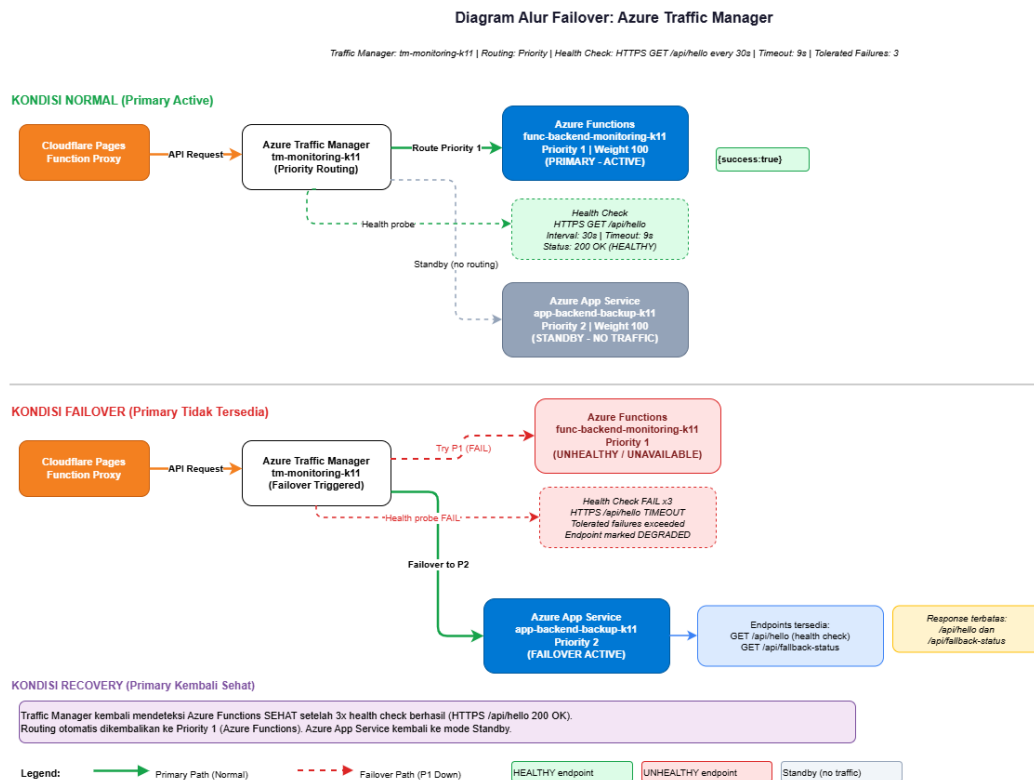
Lapisan 4 — Security, Monitoring & Optimization (Kanan)

Kolom ini menampilkan enam layanan pendukung yang bekerja di latar belakang. Azure Key Vault menyimpan semua rahasia sistem (connection string, kunci JWT). Application Insights merekam telemetry performa aplikasi secara real-time. Azure Monitor mengumpulkan metrik dan log dari semua layanan. Action Group & Alerts mengirimkan notifikasi email saat terjadi anomali. Diagnostic Settings mengirimkan semua log ke Log Analytics Workspace. Storage Lifecycle Policy secara otomatis menurunkan tier penyimpanan berkas lama untuk menghemat biaya.

Panel Bawah — Ringkasan Dua Alur Data Utama

Panel kecil di bagian bawah diagram merangkum dua skenario pemrosesan data: Instant Analysis Flow (POST /api/analyze) yang memproses data di dalam memori tanpa menyimpannya, dan Main Ingestion Flow (POST /api/upload) yang menyimpan data secara permanen ke Blob Storage lalu ke Cosmos DB melalui proses Blob Trigger.

Alur Failover Azure Traffic Manager



Gambar 3.3 — Diagram Alur Failover: Azure Traffic Manager dengan Konfigurasi Priority Routing

Gambaran Umum Diagram

Diagram ini menjelaskan secara detail bagaimana mekanisme pemulihan bencana (failover) bekerja pada sistem. Diagram dibagi menjadi tiga zona yang dibaca dari atas ke bawah: Kondisi Normal (hijau), Kondisi Failover (merah), dan Kondisi Recovery (biru muda).

Zona Atas — Kondisi Normal (Primary Active)

Pada kondisi normal, Cloudflare Pages Function Proxy mengirimkan permintaan API ke Azure Traffic Manager. Traffic Manager melihat bahwa Azure Functions (Priority 1 / Weight 100) dalam keadaan HEALTHY, sehingga seluruh permintaan diteruskan ke sana. Health Check berjalan setiap 30 detik via HTTPS GET /api/hello dengan timeout 9 detik dan mengembalikan status 200 OK. Azure App Service (Priority 2) terlihat berwarna abu-abu dengan label 'STANDBY - NO TRAFFIC', menandakan ia siaga tetapi tidak menerima permintaan apa pun.

Zona Tengah — Kondisi Failover (Primary Tidak Tersedia)

Ketika Azure Functions berhenti merespons, Traffic Manager mencatat kegagalan health check. Garis merah putus-putus pada diagram merepresentasikan upaya Traffic Manager mencapai Azure Functions yang kini berstatus UNHEALTHY. Setelah kegagalan health check terjadi 3 kali berturut-turut (total deteksi sekitar 90 detik), Traffic Manager menandai endpoint Priority 1 sebagai DEGRADED dan mengaktifkan failover ke Priority 2.

Anak panah hijau tebal bertuliskan 'Failover to P2' menandai momen perpindahan rute ke Azure App Service yang kini berstatus FAILOVER ACTIVE. Panel kuning di kanan bawah menunjukkan bahwa respons yang tersedia sangat terbatas: hanya /api/hello dan /api/fallback-status.

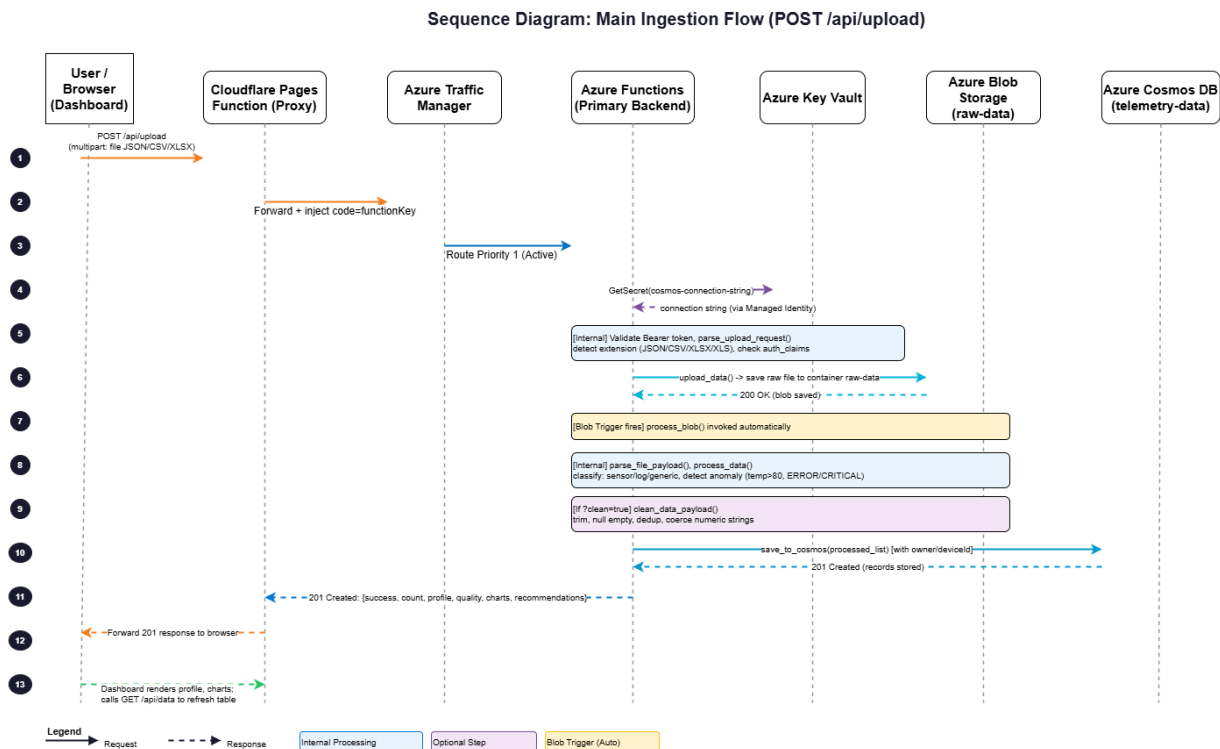
Zona Bawah — Kondisi Recovery (Primary Kembali Sehat)

Setelah Azure Functions berhasil dipulihkan, Traffic Manager kembali mendeteksi respons sehat (HTTPS /api/hello 200 OK) sebanyak 3 kali berturut-turut. Secara otomatis, Traffic Manager memulihkan ruting ke Priority 1 (Azure Functions) dan Azure App Service kembali ke mode Standby. Seluruh proses ini terjadi tanpa intervensi manual dari administrator sistem.

Penjelasan Langkah demi Langkah

1	Traffic Manager melakukan health check ke /api/hello setiap 30 detik.
2	Kegagalan health check 3x berturut-turut memicu status DEGRADED pada endpoint utama.
3	Traffic Manager secara otomatis mengalihkan semua permintaan ke App Service (Priority 2).
4	Pengguna menerima respons dari backup backend dengan endpoint terbatas.
5	Setelah backend utama pulih (3x health check sukses), ruting otomatis kembali ke Priority 1.

Sequence Diagram: Main Ingestion Flow (POST /api/upload)



Gambar 3.5 — Sequence Diagram Alur Pengunggahan & Pemrosesan Data Utama (Main Ingestion Flow)

Gambaran Umum Diagram

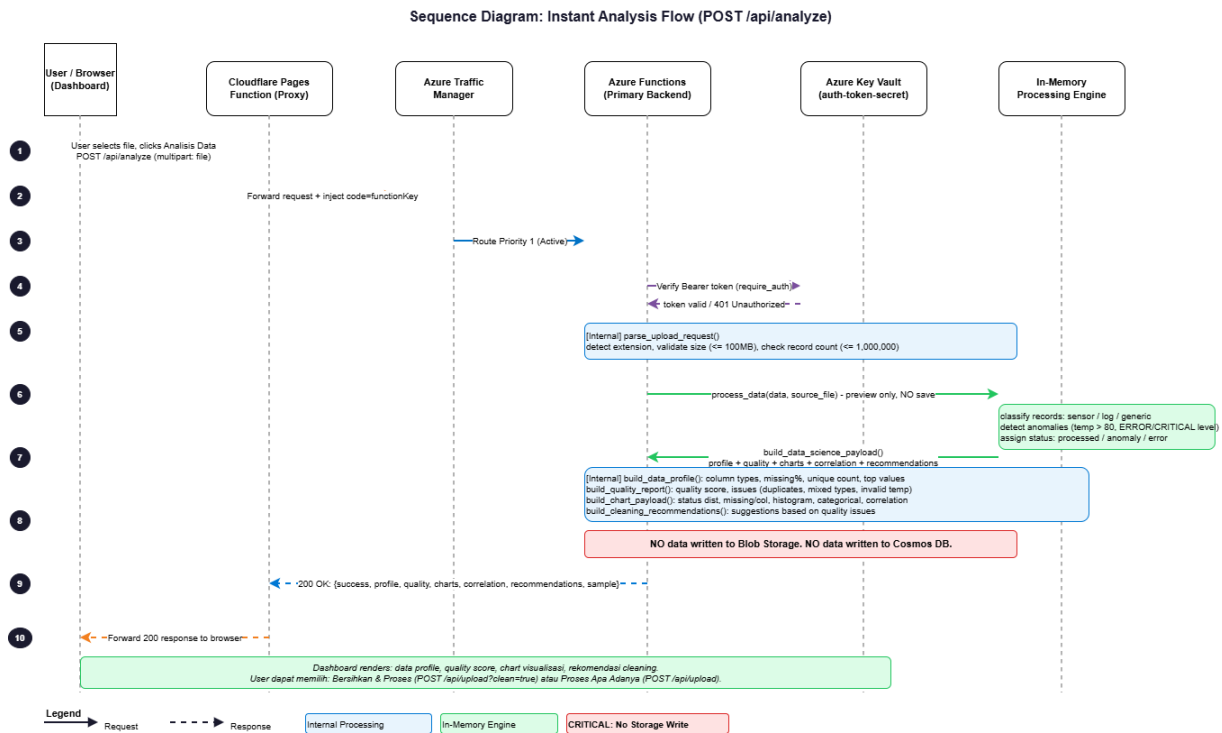
Diagram urutan (sequence diagram) ini melacak perjalanan sebuah berkas data dari saat pengguna menekan tombol 'Upload' di browser hingga data tersebut berhasil tersimpan permanen di Cosmos DB. Setiap kolom mewakili satu komponen sistem (aktor), dan setiap anak panah adalah pesan atau tindakan yang berpindah antar komponen. Waktu mengalir dari atas (langkah 1) ke bawah (langkah 13).

Penjelasan Langkah demi Langkah

1	Pengguna memilih berkas (JSON/CSV/XLSX) di dashboard dan mengirimkan permintaan POST /api/upload dengan berkas terlampir (multipart form-data).
2	Cloudflare Pages Function Proxy menerima permintaan, menyisipkan code=functionKey secara rahasia di URL, lalu meneruskan ke Traffic Manager.
3	Traffic Manager memverifikasi kesehatan endpoint dan meneruskan permintaan ke Azure Functions (Priority 1 - Active).
4	Azure Functions mengambil connection string Cosmos DB dari Azure Key Vault menggunakan Managed Identity (tanpa password manual).
5	Azure Functions memvalidasi token JWT di header Authorization, mem-parsing berkas, mendeteksi ekstensi file, dan memverifikasi klaim autentikasi pengguna.
6	Salinan berkas mentah diunggah ke kontainer 'raw-data' di Azure Blob Storage sebagai arsip. Storage merespons 200 OK.

7	Blob Trigger aktif: penyimpanan berkas di Blob Storage secara otomatis memicu fungsi process_blob() tanpa perintah HTTP tambahan.
8	process_blob() mem-parsing isi berkas, mengklasifikasikan rekaman (sensor/log/generic), dan mendeteksi anomali (suhu > 80, level ERROR/CRITICAL).
9	Jika parameter ?clean=true aktif, mesin pembersih data berjalan: pemangkasan teks, konversi null, penghapusan duplikat, normalisasi tipe data numerik dan boolean.
10	Semua rekaman yang telah diproses disimpan ke kontainer 'telemetry-data' di Cosmos DB dengan metadata owner dan deviceId. Cosmos DB merespons 201 Created.
11	Azure Functions mengirimkan respons '201 Created' berisi laporan lengkap: jumlah rekaman, profil data, skor kualitas, grafik, dan rekomendasi pembersihan.
12	Cloudflare Pages Function Proxy meneruskan respons 201 tersebut kembali ke browser pengguna.
13	Dashboard di browser merender grafik profil data dan memanggil GET /api/data untuk memperbarui tabel riwayat telemetry.

Sequence Diagram: Instant Analysis Flow (POST /api/analyze)



Gambar 3.6 — Sequence Diagram Alur Analisis Data Instan tanpa Penyimpanan (Instant Analysis Flow)

Gambaran Umum Diagram

Diagram ini menggambarkan alur analisis data secara instan dan aman: pengguna dapat memeriksa kualitas sebuah berkas data tanpa sedetik pun data tersebut tersimpan di server. Alur ini lebih pendek dari Diagram 3 karena tidak melibatkan Blob Storage atau Cosmos DB sama sekali. Kotak merah bertuliskan 'NO data written to Blob Storage. NO data written to Cosmos DB.' adalah jaminan teknis paling penting dalam diagram ini.

Perbedaan Utama dengan Main Ingestion Flow (Diagram 3)

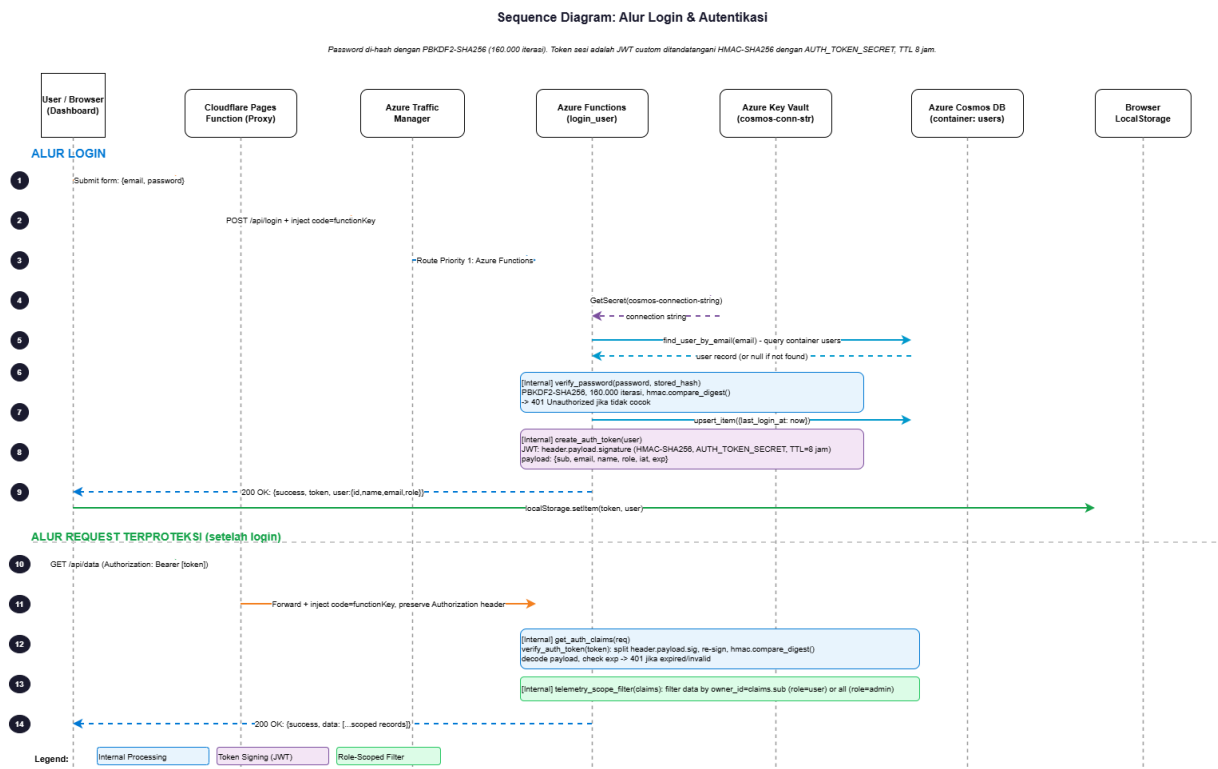
Pada Instant Analysis Flow, tidak ada panah yang mengarah ke Azure Blob Storage atau Azure Cosmos DB. Ini berarti tidak ada data yang meninggalkan memori Azure Functions. Alur ini dirancang untuk pengguna yang ingin 'mencicipi' atau menginspeksi kualitas data sebelum memutuskan apakah akan menyimpannya atau tidak. Setelah melihat hasil analisis, pengguna dapat memilih tindakan lanjutan: menyimpan data apa adanya (POST /api/upload) atau menyimpan setelah dibersihkan (POST /api/upload?clean=true).

Penjelasan Langkah demi Langkah

1	Pengguna memilih berkas dan menekan tombol 'Analisis Data' di dashboard. Browser mengirimkan POST /api/analyze dengan berkas terlampir.
2	Cloudflare Pages Function Proxy meneruskan permintaan dan menyisipkan code=functionKey ke URL tujuan.
3	Traffic Manager meneruskan ke Azure Functions (Priority 1 - Active).
4	Azure Functions memverifikasi Bearer Token dengan Azure Key Vault (auth-token-secret). Jika token tidak valid, langsung dikembalikan 401 Unauthorized.

5	Azure Functions mem-parsing berkas, mendeteksi ekstensi, memvalidasi ukuran (maks 100 MB) dan jumlah rekaman (maks 1.000.000).
6	Data dikirimkan ke In-Memory Processing Engine: proses analisis berlangsung sepenuhnya di dalam RAM Azure Functions tanpa menulis apa pun ke storage.
7	In-Memory Engine mengklasifikasikan rekaman, mendeteksi anomali, dan menghitung statistik kualitas, lalu mengembalikan hasilnya ke Azure Functions.
8	Azure Functions membangun laporan lengkap: profil kolom, skor kualitas, grafik distribusi, matriks korelasi, dan rekomendasi cleaning. Jaminan: TIDAK ADA data ditulis ke Blob Storage maupun Cosmos DB.
9	Azure Functions mengirimkan respons '200 OK' berisi profil, kualitas, grafik, korelasi, rekomendasi, dan data sampel ke Cloudflare Proxy.
10	Cloudflare Proxy meneruskan respons 200 ke browser. Dashboard merender hasil analisis secara visual.

Sequence Diagram: Alur Login & Autentikasi



Gambar 3.7 — Sequence Diagram Alur Login, Penerbitan Token JWT, dan Akses Request Terproteksi

Gambaran Umum Diagram

Diagram ini adalah yang paling kompleks karena mencakup dua alur sekaligus dalam satu gambar: Alur Login (langkah 1-9) dan Alur Request Terproteksi pasca-login (langkah 10-14). Diagram ini menjelaskan secara detail bagaimana sistem memastikan hanya pengguna yang terautentikasi dan berwenang yang dapat mengakses data mereka masing-masing.

Penjelasan Langkah demi Langkah

1	[ALUR LOGIN] Pengguna mengisi email dan password di formulir login, lalu menekan tombol Submit.
2	Browser mengirimkan POST /api/login. Cloudflare Proxy meneruskan permintaan dan menyisipkan Function Key.
3	Traffic Manager meneruskan ke Azure Functions (fungsi login_user).
4	Azure Functions mengambil connection string Cosmos DB dari Key Vault menggunakan Managed Identity.
5	Azure Functions mencari rekaman pengguna berdasarkan email di container 'users' pada Cosmos DB. Jika tidak ditemukan, proses berhenti di sini.
6	Azure Functions menjalankan verify_password(): membandingkan hash PBKDF2-SHA256 (160.000 iterasi) dari password yang dikirim dengan hash tersimpan di Cosmos DB menggunakan hmac.compare_digest() yang tahan serangan timing.

7	Jika hash cocok, Azure Functions memperbarui field 'last_login_at' pada dokumen pengguna di Cosmos DB.
8	Azure Functions membuat dan menandatangani token JWT (HMAC-SHA256) menggunakan AUTH_TOKEN_SECRET dari Key Vault. Token berisi payload: sub (ID pengguna), email, name, role, iat (waktu dibuat), exp (waktu kedaluwarsa: 8 jam).
9	Respons '200 OK' dikirimkan berisi token JWT dan data profil pengguna. Browser menyimpan token ke localStorage.
10	[ALUR REQUEST TERPROTEKSI] Pengguna meminta data (misal: GET /api/data). Browser menyertakan token JWT di header Authorization: Bearer [token].
11	Cloudflare Proxy meneruskan permintaan beserta header Authorization (tidak dihapus) dan menyisipkan Function Key.
12	Azure Functions menjalankan get_auth_claims(): memisahkan header.payload.signature dari token, menandatangani ulang payload dengan AUTH_TOKEN_SECRET, membandingkan tanda tangan menggunakan hmac.compare_digest(), dan memeriksa waktu kedaluwarsa.
13	Jika token valid, Azure Functions menjalankan telemetry_scope_filter(): pengguna berperan 'user' hanya melihat data miliknya (owner_id = claims.sub), pengguna berperan 'admin' melihat semua data.
14	Respons '200 OK' berisi data yang telah disaring sesuai kewenangan pengguna dikembalikan ke browser.

Dokumen ini diterbitkan oleh Kelompok 11 sebagai bagian dari Laporan Teknis Akhir mata kuliah Cloud Computing 2026. Semua diagram yang dijelaskan di atas tersedia dalam format PNG dan DrawIO di dalam direktori docs/evidence pada repositori proyek.